

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO3010 - Microprocessors-Microcontrollers

LAB REPORT

Lab 1 - LED Animations

Advisor(s): Phan Văn Sỹ

Student(s): Nguyễn Đức Phi Long ID 2452689

HO CHI MINH CITY, SEPTEMBER 2026



Contents

1	Github	4
2	Exercise and Report	4
2.1	Exercise 1	4
2.2	Exercise 2	6
2.3	Exercise 3	7
2.4	Exercise 4	10
2.5	Exercise 5	13
2.6	Exercise 6	18
2.7	Exercise 7	21
2.8	Exercise 8	22
2.9	Exercise 9	22
2.10	Exercise 10	22

1 Github

Link for the every exercises in Lab 1: [Link](#)

2 Exercise and Report

2.1 Exercise 1

From the simulation on Proteus, one more LED is connected to pin **PA6** of the STM32 (negative pin of the LED is connected to PA6). The component suggested in this exercise is **LED-YELLOW**, which can be found from the device list.

In this exercise, the status of two LEDs are switched every 2 seconds, as demonstrated in the figure bellow.

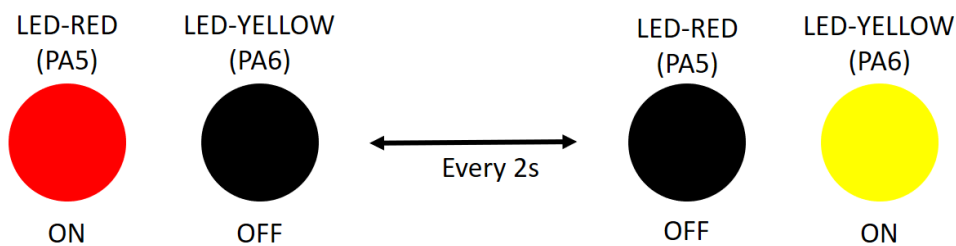


Figure 2.1: *State transitions for 2 LEDs*

Report 1: Depict the schematic from Proteus simulation in this report. The caption of the figure is a downloadable link to the Proteus project file (e.g. a github link).

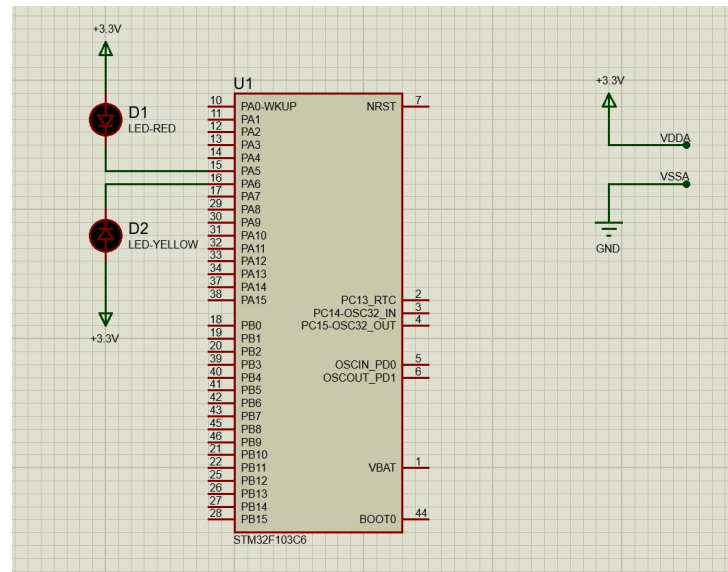


Figure 2.2: *Proteus schematic for 2 LEDs*

Report 2: Present the source code in the infinite while of your project. If a user-defined functions is used, it is required to present in this part. A brief description can be added for this function (e.g. using comments). A template to present your source code is presented bellow.

Listing 2.1: Exercise 01 source code in while loop

```
1 while (1)
2 {
3     HAL_GPIO_WritePin(LED_RED_GPIO_Port , LED_RED_Pin ,
4         GPIO_PIN_RESET);
5     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port , LED_YELLOW_Pin ,
6         GPIO_PIN_SET);
7     HAL_Delay(2000);
8
9     HAL_GPIO_WritePin(LED_RED_GPIO_Port , LED_RED_Pin ,
10        GPIO_PIN_SET);
11    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port , LED_YELLOW_Pin ,
12        GPIO_PIN_RESET);
13    HAL_Delay(2000);
14 }
```

2.2 Exercise 2

Extend the first exercise to simulate the behavior of a traffic light. A third LED, named **LED-GREEN** is added to the system, which is connected to **PA7**. A cycle in this traffic light is 5 seconds for the RED, 2 seconds for the YELLOW and 3 seconds for the GREEN. The LED-GREEN is also controlled by its negative pin.

Similarly, the report in this exercise includes the schematic of your circuit and a your source code in the while loop.

Report 1: Present the schematic.

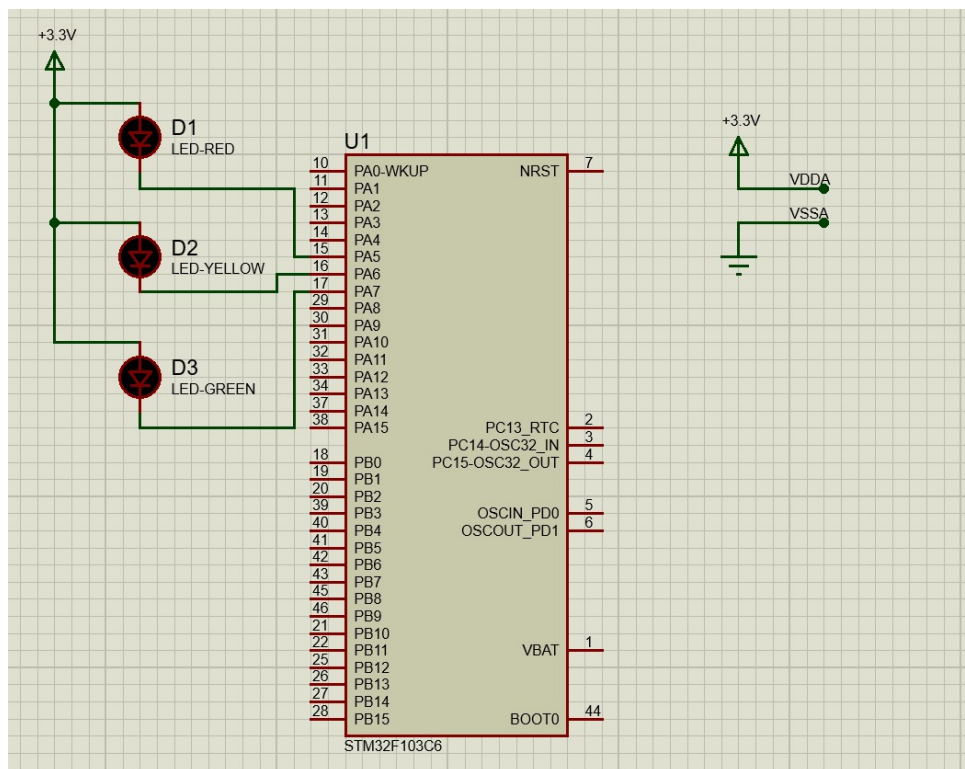


Figure 2.3: *Proteus schematic for 3 LEDs*

Report 2: Present the source code in while.

Listing 2.2: Exercise 02 source code in while loop

```
1 while (1)
2 {
```

```
3  HAL_GPIO_WritePin(LED_RED_GPIO_Port , LED_RED_Pin ,
    GPIO_PIN_RESET);
4  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port , LED_YELLOW_Pin ,
    GPIO_PIN_SET);
5  HAL_GPIO_WritePin(LED_GREEN_GPIO_Port , LED_GREEN_Pin ,
    GPIO_PIN_SET);
6  HAL_Delay(5000);
7
8  HAL_GPIO_WritePin(LED_RED_GPIO_Port , LED_RED_Pin ,
    GPIO_PIN_SET);
9  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port , LED_YELLOW_Pin ,
    GPIO_PIN_SET);
10 HAL_GPIO_WritePin(LED_GREEN_GPIO_Port , LED_GREEN_Pin ,
    GPIO_PIN_RESET);
11 HAL_Delay(3000);
12
13 HAL_GPIO_WritePin(LED_RED_GPIO_Port , LED_RED_Pin ,
    GPIO_PIN_SET);
14 HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port , LED_YELLOW_Pin ,
    GPIO_PIN_RESET);
15 HAL_GPIO_WritePin(LED_GREEN_GPIO_Port , LED_GREEN_Pin ,
    GPIO_PIN_SET);
16 HAL_Delay(2000);
17 }
```

2.3 Exercise 3

Extend to the 4-way traffic light. Arrange 12 LEDs in a nice shape to simulate the behaviors of a traffic light. A reference design can be found in the figure bellow.

Report:

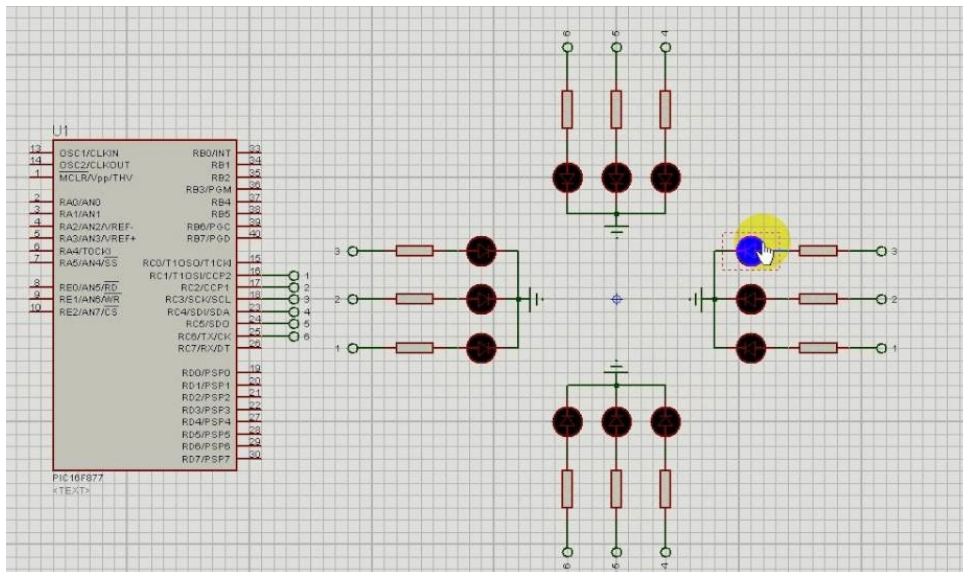


Figure 2.4: Reference design for a 4 way traffic light

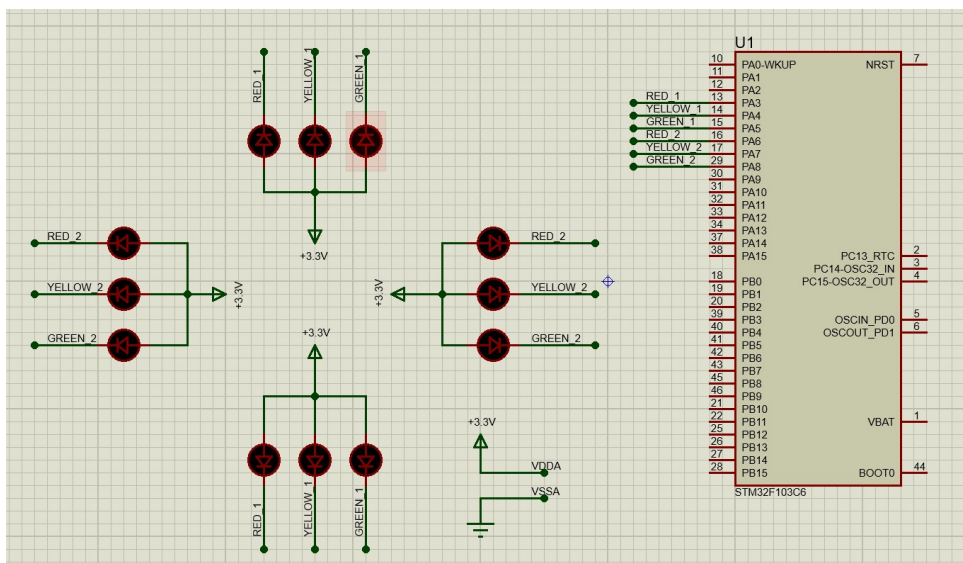


Figure 2.5: Proteus schematic for a 4-way traffic light

- Due to the increasing of number of LEDs, we will use arrays to handle them, or struct if you prefer.

Listing 2.3: Exercise 03 source code

```
1 MX_GPIO_Init();
2 /* USER CODE BEGIN 2 */
3
4 GPIO_TypeDef* ledPorts[] = { RED_1_GPIO_Port,
```

```
YELLOW_1_GPIO_Port, GREEN_1_GPIO_Port,
5         RED_2_GPIO_Port,
        YELLOW_2_GPIO_Port,
        GREEN_2_GPIO_Port };
6 uint16_t ledPins[] = { RED_1_Pin, YELLOW_1_Pin,
        GREEN_1_Pin,
7         RED_2_Pin, YELLOW_2_Pin,
        GREEN_2_Pin };
8
9 GPIO_PinState phase1[] = { SET,    SET,    RESET, RESET,
        SET,    SET    };
10 GPIO_PinState phase2[] = { SET,    RESET, SET,    RESET,
        SET,    SET    };
11 GPIO_PinState phase3[] = { RESET, SET,    SET,    SET,
        SET,    RESET };
12 GPIO_PinState phase4[] = { RESET, SET,    SET,    SET,
        RESET, SET    };
13
14 uint32_t delays[] = { 3000, 2000, 3000, 2000 };
15 GPIO_PinState* phases[] = { phase1, phase2, phase3,
        phase4 };
16
17 /* USER CODE END 2 */
18
19 /* Infinite loop */
20 /* USER CODE BEGIN WHILE */
21 while (1)
22 {
23     for (int p = 0; p < 4; p++)
24     {
25         for (int i = 0; i < 6; i++)
26         {
27             HAL_GPIO_WritePin(ledPorts[i], ledPins[i], phases
                [p][i]);
28         }
```

```

29     HAL_Delay(delays[p]);
30 }
31 }
32 /* USER CODE END WHILE */

```

2.4 Exercise 4

Add **only one 7 led segment** to the schematic in Exercise 3. This component can be found in Proteus by the keyword **7SEG-COM-ANODE**. For this device, the common pin should be connected to the power supply and other pins are supposed to be connected to PB0 to PB6. Therefore, to turn-on a segment in this 7SEG, the STM32 pin should be in logic 0 (0V).

Implement a function named **display7SEG(int num)**. The input for this function is from 0 to 9 and the outputs are listed as following:

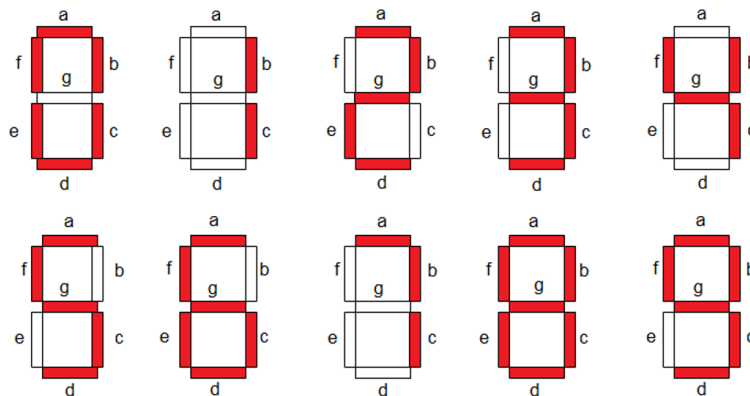


Figure 2.6: *Display a number on 7 segment LED*

This function is invoked in the while loop for testing as following:

Listing 2.4: An example for your source code

```

1 int counter = 0;
2 while (1){
3     if(counter >= 10) counter = 0;
4     display7SEG(counter++);
5     HAL_Delay(1000);
6 }

```

7 }

Report 1: Present the schematic.

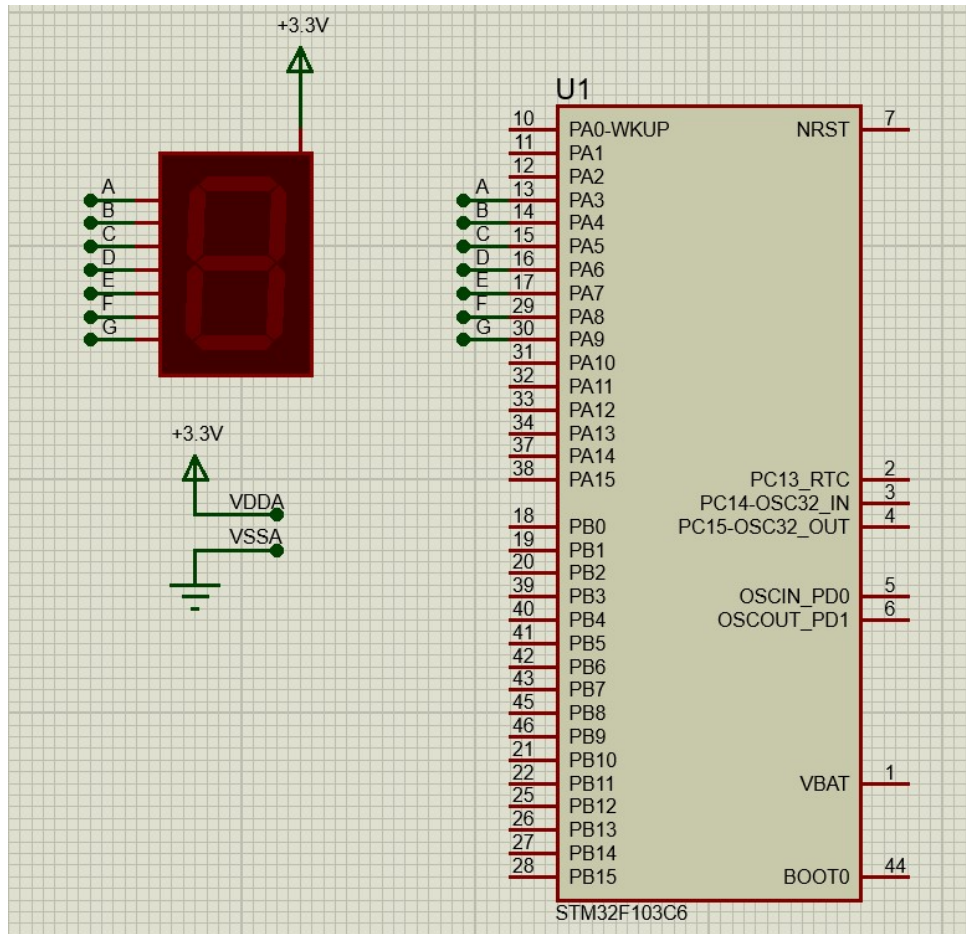


Figure 2.7: Proteus schematic for a 7-segment LED

Report 2: Present the source code for display7SEG function.

Listing 2.5: Exercise 04 source code

```
1 MX_GPIO_Init();
2 /* USER CODE BEGIN 2 */
3
4 typedef struct
5 {
6     GPIO_TypeDef *port;
7     uint16_t pin;
```

```
8 } SegPin;
9
10 SegPin segments[] =
11 {
12     { A_GPIO_Port, A_Pin },
13     { B_GPIO_Port, B_Pin },
14     { C_GPIO_Port, C_Pin },
15     { D_GPIO_Port, D_Pin },
16     { E_GPIO_Port, E_Pin },
17     { F_GPIO_Port, F_Pin },
18     { G_GPIO_Port, G_Pin },
19 };
20
21 GPIO_PinState digitPatterns[10][7] =
22 {
23     { RESET, RESET, RESET, RESET, RESET, RESET, SET }, //
24     0
25     { SET, RESET, RESET, SET, SET, SET, SET }, //
26     1
27     { RESET, RESET, SET, RESET, RESET, SET, RESET }, //
28     2
29     { RESET, RESET, RESET, RESET, SET, SET, RESET }, //
30     3
31     { SET, RESET, RESET, SET, SET, RESET, RESET }, //
32     4
33     { RESET, SET, RESET, RESET, SET, RESET, RESET }, //
34     5
35     { RESET, SET, RESET, RESET, RESET, RESET, RESET }, //
36     6
37     { RESET, RESET, RESET, SET, SET, SET, SET }, //
38     7
39     { RESET, RESET, RESET, RESET, RESET, RESET, RESET }, //
40     8
41     { RESET, RESET, RESET, RESET, SET, RESET, RESET }, //
42     9
```

```
33 };  
34  
35 void display7SEG(int num)  
36 {  
37     for (int i = 0; i < 7; i++)  
38     {  
39         HAL_GPIO_WritePin(segments[i].port, segments[i].pin,  
40             digitPatterns[num][i]);  
41     }  
42  
43     int counter = 0;  
44  
45     /* USER CODE END 2 */  
46  
47     /* Infinite loop */  
48     /* USER CODE BEGIN WHILE */  
49     while (1)  
50     {  
51         if(counter >= 10) counter = 0;  
52         display7SEG(counter++);  
53         HAL_Delay(1000);  
54     }  
55     /* USER CODE END WHILE */
```

2.5 Exercise 5

Integrate the 7SEG-LED to the 4 way traffic light. In this case, the 7SEG-LED is used to display countdown value.

In this exercise, only source code is required to present. The function display7SEG in previous exercise can be re-used.

Report:

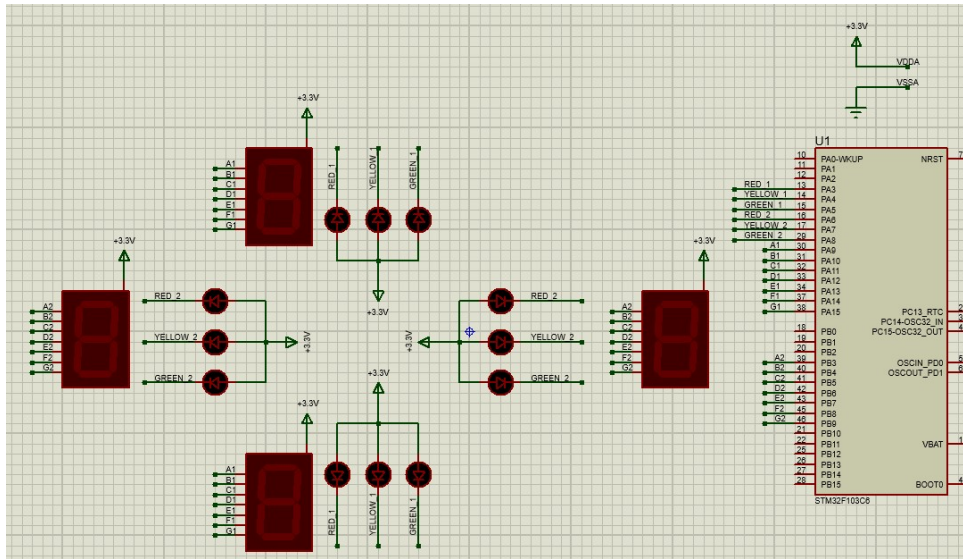


Figure 2.8: Proteus schematic for a 4-way traffic with 7-segment LED counters

Listing 2.6: Exercise 05 source code

```

1 MX_GPIO_Init();
2 /* USER CODE BEGIN 2 */
3
4 GPIO_TypeDef* ledPorts[] = { RED_1_GPIO_Port,
5                               YELLOW_1_GPIO_Port, GREEN_1_GPIO_Port,
6                               RED_2_GPIO_Port,
7                               YELLOW_2_GPIO_Port,
8                               GREEN_2_GPIO_Port };
9
10 uint16_t ledPins[] = { RED_1_Pin, YELLOW_1_Pin,
11                        GREEN_1_Pin,
12                        RED_2_Pin, YELLOW_2_Pin,
13                        GREEN_2_Pin };
14
15 GPIO_PinState phase1[] = { SET, SET, RESET, RESET,
16                             SET, SET }; // GREEN_1, RED_2
17 GPIO_PinState phase2[] = { SET, RESET, SET, RESET,
18                             SET, SET }; // YELLOW_1, RED_2
19 GPIO_PinState phase3[] = { RESET, SET, SET, SET,
20                             SET, RESET }; // RED_1, GREEN_2
21 GPIO_PinState phase4[] = { RESET, SET, SET, SET,

```



```
RESET, SET    }; // RED_1, YELLOW_2

13
14 void setLeds(GPIO_PinState *pattern)
15 {
16     for (int i = 0; i < 6; i++)
17         HAL_GPIO_WritePin(ledPorts[i], ledPins[i],
18                             pattern[i]);
19 }
20
21 typedef struct
22 {
23     GPIO_TypeDef *port;
24     uint16_t pin;
25 } SegPin;
26
27 SegPin segments_1[] =
28 {
29     { A1_GPIO_Port, A1_Pin },
30     { B1_GPIO_Port, B1_Pin },
31     { C1_GPIO_Port, C1_Pin },
32     { D1_GPIO_Port, D1_Pin },
33     { E1_GPIO_Port, E1_Pin },
34     { F1_GPIO_Port, F1_Pin },
35     { G1_GPIO_Port, G1_Pin },
36 };
37
38 SegPin segments_2[] =
39 {
40     { A2_GPIO_Port, A2_Pin },
41     { B2_GPIO_Port, B2_Pin },
42     { C2_GPIO_Port, C2_Pin },
43     { D2_GPIO_Port, D2_Pin },
44     { E2_GPIO_Port, E2_Pin },
45     { F2_GPIO_Port, F2_Pin },
46     { G2_GPIO_Port, G2_Pin },
```

```
46     };
47
48     GPIO_PinState digitPatterns[10][7] =
49     {
50         { RESET, RESET, RESET, RESET, RESET, RESET, SET },
51         // 0
52         { SET, RESET, RESET, SET, SET, SET, SET },
53         // 1
54         { RESET, RESET, SET, RESET, RESET, SET, RESET },
55         // 2
56         { RESET, RESET, RESET, RESET, SET, SET, RESET },
57         // 3
58         { SET, RESET, RESET, SET, SET, RESET, RESET },
59         // 4
60         { RESET, SET, RESET, RESET, SET, RESET, RESET },
61         // 5
62         { RESET, SET, RESET, RESET, RESET, RESET, RESET },
63         // 6
64         { RESET, RESET, RESET, SET, SET, SET, SET },
65         // 7
66         { RESET, RESET, RESET, RESET, RESET, RESET, RESET },
67         // 8
68         { RESET, RESET, RESET, RESET, SET, RESET, RESET },
69         // 9
70     };
71
72     void display7SEG(SegPin *segs, int num)
73     {
74         if (num < 0 || num > 9) return;
75         for (int i = 0; i < 7; i++)
76             HAL_GPIO_WritePin(segs[i].port, segs[i].pin,
77                               digitPatterns[num][i]);
78     }
79
80     int counter_red = 0;
```

```
70
71 /* USER CODE END 2 */
72
73 /* Infinite loop */
74 /* USER CODE BEGIN WHILE */
75     while (1)
76     {
77         counter_red = 5;
78         setLeds(phase1);           // GREEN 1, RED 2
79         for (int i = 3; i >= 1; i--)
80         {
81             display7SEG(segments_1, i);
82             display7SEG(segments_2, counter_red--);
83             HAL_Delay(1000);
84         }
85
86         setLeds(phase2);           // YELLOW 1, RED 2
87         for (int i = 2; i >= 1; i--)
88         {
89             display7SEG(segments_1, i);
90             display7SEG(segments_2, counter_red--);
91             HAL_Delay(1000);
92         }
93
94         counter_red = 5;
95         setLeds(phase3);           // RED 1, GREEN 2
96         for (int i = 3; i >= 1; i--)
97         {
98             display7SEG(segments_2, i);
99             display7SEG(segments_1, counter_red--);
100            HAL_Delay(1000);
101        }
102
103        setLeds(phase4);           // RED 1, GREEN 2
104        for (int i = 2; i >= 1; i--)
```

```
105     {  
106         display7SEG(segments_2 , i);  
107         display7SEG(segments_1 , counter_red--);  
108         HAL_Delay(1000);  
109     }  
110 }
```

2.6 Exercise 6

In this exercise, a new Proteus schematic is designed to simulate an analog clock, with 12 different number. The connections for 12 LEDs are supposed from PA4 to PA15 of the STM32. The arrangement of 12 LEDs is depicted as follows.

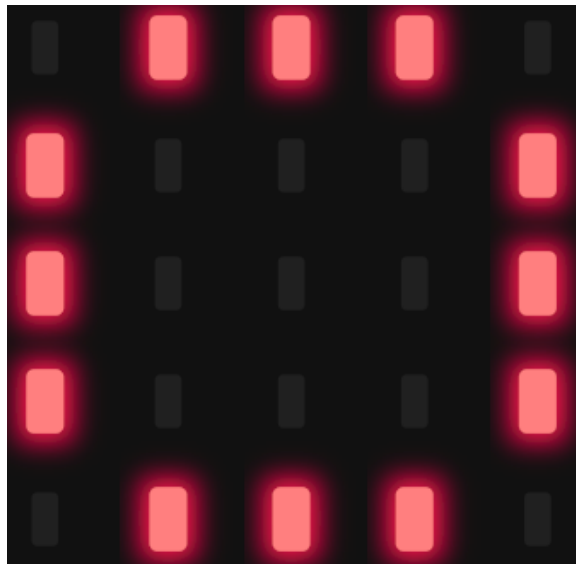


Figure 2.9: 12 LEDs for an analog clock

Report 1: Present the schematic.

****Note: this schematic will be used for Exercise 6-10, along with its code structure.***

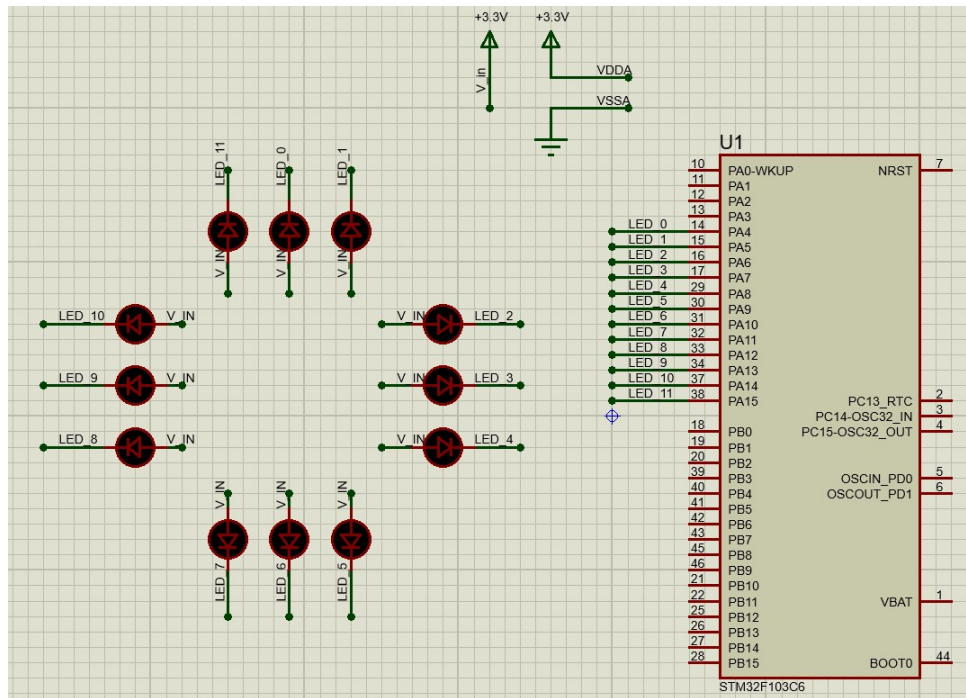


Figure 2.10: Proteus schematic for clock by using 12 red LEDs

Report 2: Implement a simple program to test the connection of every single LED. This testing program should turn every LED in a sequence.

Listing 2.7: Exercise 06 source code

```
1 MX_GPIO_Init();
2 /* USER CODE BEGIN 2 */
3
4 typedef struct
5 {
6     GPIO_TypeDef *port;
7     uint16_t pin;
8 } LedPin;
9
10 LedPin clock[] =
11 {
12     { LED_0_GPIO_Port, LED_0_Pin },
13     { LED_1_GPIO_Port, LED_1_Pin },
14     { LED_2_GPIO_Port, LED_2_Pin },
15     { LED_3_GPIO_Port, LED_3_Pin },
```

```
16 { LED_4_GPIO_Port, LED_4_Pin },
17 { LED_5_GPIO_Port, LED_5_Pin },
18 { LED_6_GPIO_Port, LED_6_Pin },
19 { LED_7_GPIO_Port, LED_7_Pin },
20 { LED_8_GPIO_Port, LED_8_Pin },
21 { LED_9_GPIO_Port, LED_9_Pin },
22 { LED_10_GPIO_Port, LED_10_Pin },
23 { LED_11_GPIO_Port, LED_11_Pin },
24 };
25
26 GPIO_PinState state[12][12] =
27 {
28     {RESET, SET, SET, SET, SET, SET, SET, SET, SET, SET, SET, SET,
29         SET}, // 0 & 12
30     {SET, RESET, SET, SET, SET, SET, SET, SET, SET, SET, SET, SET,
31         SET}, // 1
32     {SET, SET, RESET, SET, SET, SET, SET, SET, SET, SET, SET, SET,
33         SET}, // 2
34     {SET, SET, SET, RESET, SET, SET, SET, SET, SET, SET, SET, SET,
35         SET}, // 3
36     {SET, SET, SET, SET, RESET, SET, SET, SET, SET, SET, SET, SET,
37         SET}, // 4
38     {SET, SET, SET, SET, SET, RESET, SET, SET, SET, SET, SET, SET,
39         SET}, // 5
40     {SET, SET, SET, SET, SET, SET, RESET, SET, SET, SET, SET, SET,
41         SET}, // 6
42     {SET, SET, SET, SET, SET, SET, SET, RESET, SET, SET, SET, SET,
43         SET}, // 7
44     {SET, SET, SET, SET, SET, SET, SET, SET, RESET, SET, SET, SET,
45         SET}, // 8
46     {SET, SET, SET, SET, SET, SET, SET, SET, SET, RESET, SET, SET,
47         SET}, // 9
48     {SET, SET, SET, SET, SET, SET, SET, SET, SET, SET, RESET, SET,
49         SET}, // 10
50     {SET, SET, SET, SET, SET, SET, SET, SET, SET, SET, SET, SET,
51         SET}, // 11
52 }
```

```
    RESET}, // 11
40 };
41
42 void clearAllClock()
43 {
44     for (int i = 0; i < 12; i++)
45         HAL_GPIO_WritePin(clock[i].port, clock[i].pin, SET);
46 }
47
48
49
50 /* USER CODE END 2 */
51
52 /* Infinite loop */
53 /* USER CODE BEGIN WHILE */
54 while (1)
55 {
56     for (int i = 0; i < 12; i++)
57     {
58         for (int j = 0; j < 12; j++)
59             HAL_GPIO_WritePin(clock[j].port, clock[j].pin,
60                               state[i][j]);
61         HAL_Delay(1000);
62     }
63 }
```

2.7 Exercise 7

Implement a function named **clearAllClock()** to turn off all 12 LEDs. Present the source code of this function.

Listing 2.8: Function Implementation

```
1 void clearAllClock(){
2     for (int i = 0; i < 12; i++)
3         HAL_GPIO_WritePin(clock[i].port, clock[i].pin, SET);
4 }
```



2.8 Exercise 8

Implement a function named **setNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn on. Present the source code of this function.

Listing 2.9: Exercise 08 source code

```
1 void setNumberOnClock(int num)
2 {
3     HAL_GPIO_WritePin(clock[num].port, clock[num].pin,
4         GPIO_PIN_RESET);
5 }
```

2.9 Exercise 9

Implement a function named **clearNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn off.

Listing 2.10: Exercise 09 source code

```
1 void clearNumberOnClock(int num)
2 {
3     HAL_GPIO_WritePin(clock[num].port, clock[num].pin, SET);
4 }
```

2.10 Exercise 10

Integrate the whole system and use 12 LEDs to display a clock. At a given time, there are only 3 LEDs are turn on for hour, minute and second information.

Listing 2.11: Exercise 10 source code

```
1 MX_GPIO_Init();
2 /* USER CODE BEGIN 2 */
3
4 typedef struct
5 {
6     GPIO_TypeDef *port;
7     uint16_t pin;
```



```
8 } LedPin;
9
10 LedPin clock[] =
11 {
12     { LED_0_GPIO_Port, LED_0_Pin },
13     { LED_1_GPIO_Port, LED_1_Pin },
14     { LED_2_GPIO_Port, LED_2_Pin },
15     { LED_3_GPIO_Port, LED_3_Pin },
16     { LED_4_GPIO_Port, LED_4_Pin },
17     { LED_5_GPIO_Port, LED_5_Pin },
18     { LED_6_GPIO_Port, LED_6_Pin },
19     { LED_7_GPIO_Port, LED_7_Pin },
20     { LED_8_GPIO_Port, LED_8_Pin },
21     { LED_9_GPIO_Port, LED_9_Pin },
22     { LED_10_GPIO_Port, LED_10_Pin },
23     { LED_11_GPIO_Port, LED_11_Pin },
24 };
25
26 void clearAllClock()
27 {
28     for (int i = 0; i < 12; i++)
29         HAL_GPIO_WritePin(clock[i].port, clock[i].pin,
30                             GPIO_PIN_SET);
31 }
32
33 void setNumberOnClock(int num)
34 {
35     HAL_GPIO_WritePin(clock[num].port, clock[num].pin,
36                         GPIO_PIN_RESET);
37 }
38
39 void clearNumberOnClock(int num)
40 {
41     HAL_GPIO_WritePin(clock[num].port, clock[num].pin,
42                         GPIO_PIN_SET);
43 }
```



```
40 }
41
42 int hour    = 6;
43 int minute  = 49;
44 int second  = 50;
45
46 int prevHourPin = hour % 12;
47 int prevMinPin  = minute / 5;
48 int prevSecPin  = second / 5;
49
50 clearAllClock();
51
52 /* USER CODE END 2 */
53
54 /* Infinite loop */
55 /* USER CODE BEGIN WHILE */
56 while (1)
57 {
58     int hourPin = hour % 12;           // 0-11 directly
59     int minPin  = minute / 5;         // 0-59 -> 0-11 (each LED
        = 5 min)
60     int secPin  = second / 5;         // 0-59 -> 0-11 (each LED
        = 5 sec)
61
62     // Turn off previous clock pin (if different between [now
        ] & [prev])
63     if (second == 0 || prevHourPin != hourPin)
        clearNumberOnClock(prevHourPin);
64     if (second == 0 || prevMinPin != minPin)
        clearNumberOnClock(prevMinPin);
65     clearNumberOnClock(prevSecPin);
66
67     // Change clock pin to current time
68     setNumberOnClock(hourPin);
69     setNumberOnClock(minPin);
```

```
70     setNumberOnClock(secPin);
71
72     prevHourPin = hourPin;
73     prevMinPin  = minPin;
74     prevSecPin  = secPin;
75
76     HAL_Delay(1000);
77
78     // Second increment
79     second++;
80     if (second >= 60)
81     {
82         second = 0;
83         minute++;
84         if (minute >= 60)
85         {
86             minute = 0;
87             hour++;
88             if (hour >= 12) hour = 0;
89         }
90     }
91 }
```